

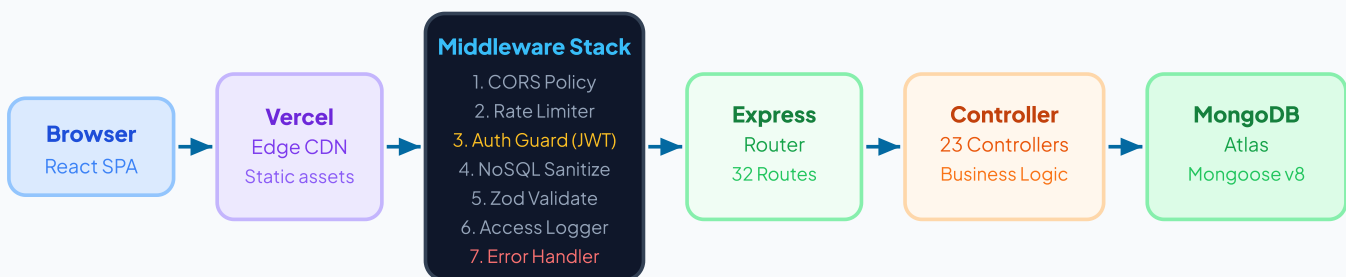
Backend API & Server Core

Node.js 22 LTS micro-monolith, Express v4 REST API, 7-layer middleware pipeline, Zod schema validation, and clean layered architecture.

Runtime: Node.js 22 LTS 32 Routes • 23 Controllers 7-Layer Middleware Pipeline

1. Full-Stack Request Architecture

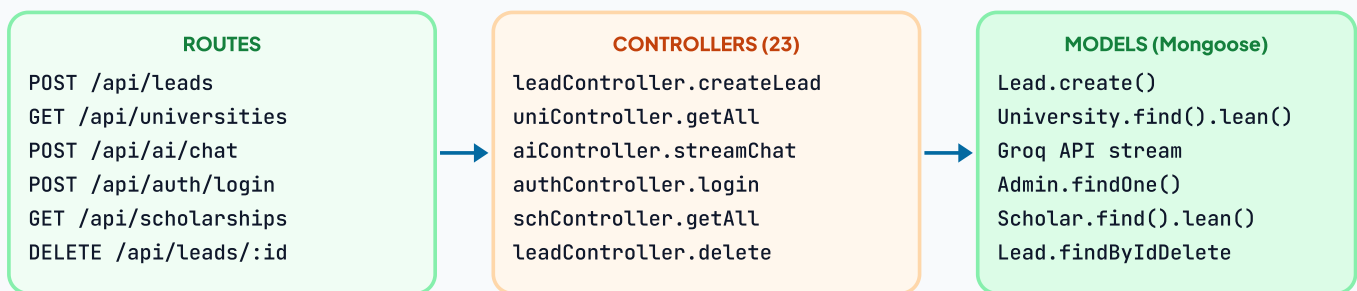
END-TO-END HTTP REQUEST LIFECYCLE — FROM BROWSER TO DATABASE



7-LAYER MIDDLEWARE PIPELINE — SECURITY-FIRST EXPRESS CONFIGURATION



EXPRESS ROUTE → CONTROLLER → MODEL MAPPING (SAMPLE ENDPOINTS)



2. Technology Decision Framework

1. Node.js + Express vs NestJS

RUNTIME

WHAT IS IT?

A JavaScript/Node.js HTTP server using the Express.js minimalist framework to handle REST API requests.

HOW DOES IT WORK & PROBLEM SOLVED?

Express's non-blocking event loop handles thousands of concurrent API requests on a single thread. The 7-layer middleware pipeline processes each request through security, authentication, validation, and logging before hitting business logic.

WHAT ALTERNATIVES EXISTED?

NestJS: Opinionated TypeScript framework with DI containers — excellent for large enterprise teams but adds significant boilerplate overhead for a 2–3 person startup. **Fastify:** 2× faster than Express, but smaller ecosystem.

WHY DID WE CHOOSE THIS APPROACH?

Maximum velocity for a startup: Express's minimal structure lets us ship features faster, and the full Node.js ecosystem (Mongoose, Groq SDK, Twilio) provides first-class integration support.

2. Zod Schema Validation vs Manual Validation

DATA SAFETY

WHAT IS IT?

TypeScript-first runtime schema validation library that rejects malformed API requests before they reach business logic.

HOW DOES IT WORK & PROBLEM SOLVED?

Without Zod, invalid data (e.g., non-numeric phone numbers, missing email fields) silently propagates into MongoDB, creating corrupt lead documents. Zod rejects invalid payloads at the middleware layer with structured 400 error responses.

WHAT ALTERNATIVES EXISTED?

Joi: Runtime-only, no TypeScript integration. **Manual if-else validation:** Repetitive, inconsistent, and error-prone.

WHY DID WE CHOOSE THIS APPROACH?

Single source of truth for API contracts: schema definitions simultaneously validate runtime data and infer TypeScript types.

💡 Senior Interview Defense: "Walk me through your backend architecture"

"UniCoach's backend is a Node.js 22 micro-monolith with 32 REST endpoints organized as a clean MVC: routes declare HTTP verbs and paths, controllers hold business logic, and Mongoose models handle persistence. Every inbound request traverses a 7-layer middleware pipeline — CORS, rate limiting, JWT auth, NoSQL injection sanitization, Zod schema validation, access logging, and a global error handler. This pipeline catches security issues, auth failures, and malformed payloads before they ever reach business logic."